

SPI مخفف (Serial Peripheral Interface) یک باس رابط است که معمولاً برای ارتباط سریال بین سیستم‌های میکروکامپیوتری و سایر دستگاه‌ها، حافظه‌ها و سنسورها استفاده می‌شود. معمولاً برای رابط‌دهی به حافظه‌های فلش، ADC، DAC، RTC، LCD، Sdcard و موارد دیگر استفاده می‌شود. SPI در اصل توسط موتورولا در دهه ۸۰ میلادی توسعه داده شد تا ارتباط سریال دوطرفه را برای استفاده در برنامه‌های سیستم‌های embedded فراهم کند.

در ارتباط معمولی SPI، حداقل باید ۲ دستگاه به باس SPI متصل باشند. یکی از آن‌ها باید مستر (master) و دیگری اساساً (slave) خواهد بود. مستر ارتباط را با تولید سیگنال ساعت سریال برای انتقال یک فریم داده آغاز می‌کند، و همزمان داده‌هایی را دریافت می‌کند. این فرآیند در هر دو حالت خواندن یا نوشتن اجرا می‌گردد.

معمولاً، SPI از طریق ۴ پین به دستگاه‌های خارجی متصل می‌شود:

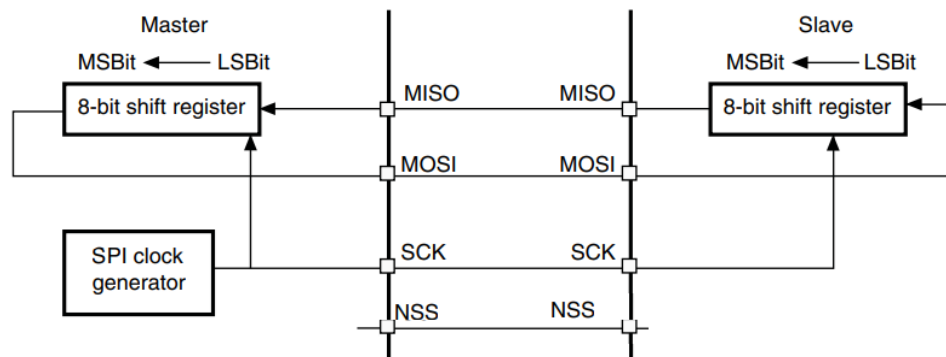
- MISO : داده‌های ورودی مستر / خروجی اسلیو. این پین می‌تواند برای انتقال داده در حالت اسلیو و دریافت داده در حالت مستر استفاده شود.

- MOSI : داده‌های خروجی مستر / ورودی اسلیو. این پین می‌تواند برای انتقال داده در حالت مستر و دریافت داده در حالت اسلیو استفاده شود.

- SCK : خروجی ساعت سریال برای مسترهای SPI و ورودی برای اسلیوهای SPI.

- NSS : انتخاب اسلیو. این پین اختیاری برای انتخاب یک دستگاه اسلیو است. این پین به عنوان «انتخاب چیپ» عمل می‌کند تا به مستر SPI اجازه دهد با اسلیوها به‌طور جداگانه ارتباط برقرار کند و از تداخل در خطوط داده جلوگیری کند.

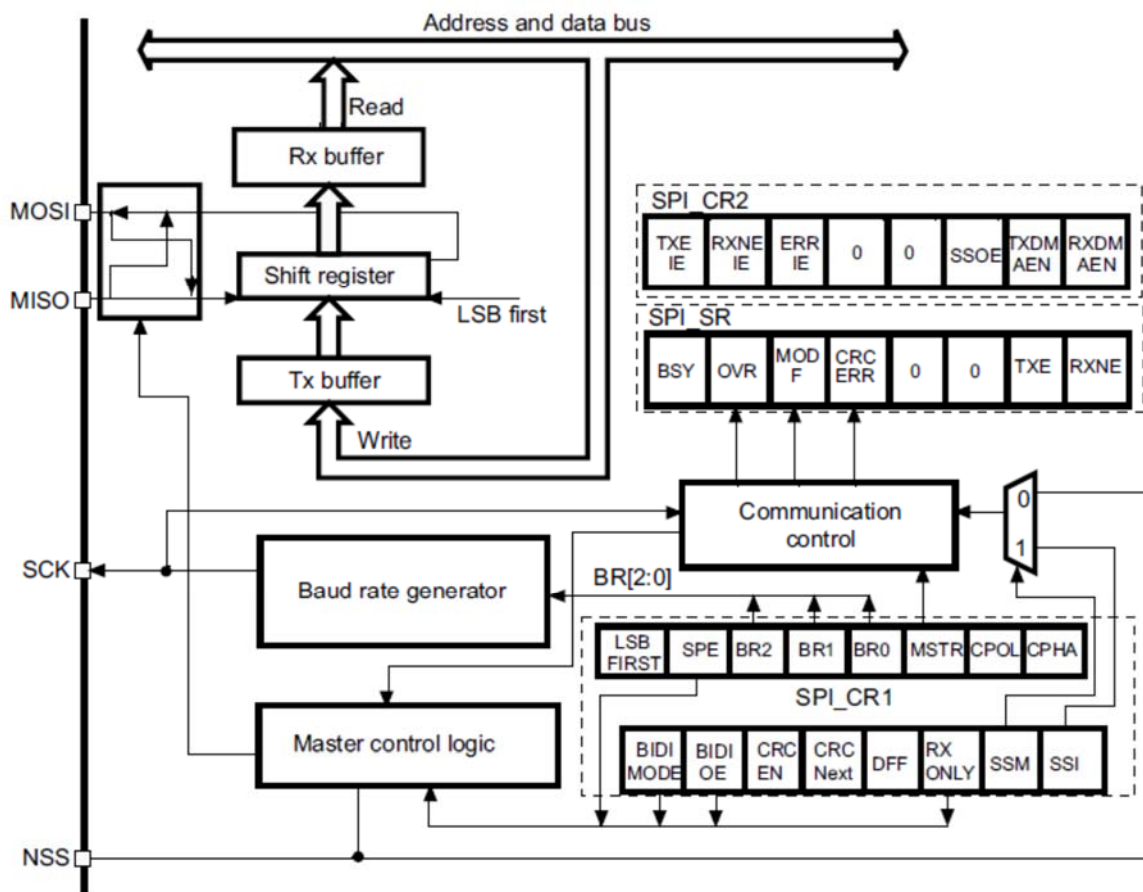
البته در برخی از ماژول‌ها از ۳ سیم برای اتصال استفاده می‌شود که ارتباط سه سیمه SPI هم نامیده می‌شود. لازم به ذکر است که بخش سخت افزار SPI به صورت مشترک با I2S کار می‌کند و در برخی پایه‌ها و وقفه‌ها و پرچم‌ها و رجیسترها مشترک هستند. نمایی از عملکرد SPI به صورت ساده در شکل ۱-۱۵ نشان داده شده است.



شکل ۱-۱۵: نمایی ساده از ارتباط SPI

15.1 سخت افزار SPI STM32

همان طور که در دیاگرام بلوکی SPI در شکل ۲-۱۵ مشاهده می کنید، یک شیفت رجیستر اصلی بین دو بافر انتقال (TX) و بافر (RX) قرار دارد. همچنین یک واحد کنترل منطقی در سمت راست وجود دارد که با سیگنال های متعددی از رجیسترهای کنترل در ارتباط است. رجیسترهای کنترل امکان پیکربندی های سخت افزاری SPI را با دستورات نرم افزاری فراهم میکند. همچنین یک رجیستر وضعیت وجود دارد که سیگنال هایی درباره تراکنش های جاری و آخرین تراکنش انجام شده و هر گونه خطای احتمالی ارائه می دهد. نرخ سیگنال ساعت SCK نیز از طریق نرم افزار قابل تنظیم است و یک منطق کنترلی خاص برای پین NSS وجود دارد.



شکل ۲-۱۵: نمایی از ساختار داخلی SPI

15.2 ۲.۲ ویژگی‌های اصلی SPI STM32

- انتقال‌های هم‌زمان دوطرفه بر روی سه خط
- انتقال‌های هم‌زمان یکطرفه بر روی دو خط
- انتخاب فرمت فریم انتقال ۸ یا ۱۶ بیتی
- عملکرد به‌عنوان مستر یا اسلیو
- قابلیت حالت چند مستر
- ۸ تقسیم‌کننده نرخ باود در حالت مستر (حداکثر $f_{PCLK}/2$)

- فرکانس در حالت اسلیو (حداکثر $f_{PCLK}/2$)
- مدیریت پین NSS توسط سخت‌افزار یا نرم‌افزار برای هر دو حالت مستر و اسلیو: تغییر دینامیک عملیات مستر/اسلیو
- قطبیت و فاز ساعت قابل برنامه‌ریزی
- ترتیب داده قابل برنامه‌ریزی با شیفت MSB-first یا LSB-first
- پرچم‌های اختصاصی انتقال و دریافت با قابلیت وقفه
- پرچم وضعیت مشغول بودن باس SPI
- ویژگی CRC سخت‌افزاری برای ارتباط مطمئن: مقدار CRC می‌تواند به عنوان آخرین بایت در حالت Tx ارسال شود - بررسی خودکار خطای CRC برای آخرین بایت دریافتی
- پرچم‌های خطای حالت مستر، سرریز و CRC با قابلیت وقفه
- بافر انتقال و دریافت ۱ بیتی با قابلیت DMA: درخواست‌های Tx و Rx

15.3 رجیسترهای SPI

برخی از رجیسترهای SPI به صورت مشترک با I2S استفاده می‌شود.

15.3.1 SPI control register 1 (SPI_CR1) (not used in I2S mode)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	DFF	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR [2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

شماره	نام	شرح ۰- غیرفعال ۱- فعال
۱۵	BIDIMODE	فعال‌سازی حالت داده دوجهتی
	۰	حالت داده یک‌جهتی با ۲ خط انتخاب شده است
	۱	حالت داده دوجهتی با ۱ خط انتخاب شده است
14	BIDIOE	فعال‌سازی خروجی در حالت دوجهتی: این بیت به همراه بیت BIDIMODE جهت انتقال را در حالت دوجهتی انتخاب می‌کند (در حالت مستر، پین MOSI استفاده می‌شود و در حالت اسلیو، پین MISO استفاده می‌شود)
	۰	خروجی غیرفعال (حالت تنها دریافت)

	۱	خروجی فعال (حالت تنها ارسال)
۱۳	CRCEN	فعال سازی محاسبه CRC سخت افزاری
	۰	محاسبه CRC غیر فعال
	۱	محاسبه CRC فعال
۱۲	CRCNEXT	انتقال CRC بعدی
	۰	مرحله داده (بدون مرحله CRC)
	۱	انتقال بعدی CRC (مرحله CRC)
۱۱	DFF	فرمت فریم داده
	۰	فرمت فریم داده ۸ بیتی
	۱	فرمت فریم داده ۱۶ بیتی
10	RXONLY	دریافت فقط
	0	دوطرفه کامل (ارسال و دریافت)
	1	خروجی غیر فعال (حالت دریافت فقط)
9	SSM	مدیریت نرم افزاری اسلیو
۸	SSI	انتخاب اسلیو داخلی این بیت فقط زمانی اثر دارد که بیت SSM تنظیم شده باشد. مقدار این بیت به پین NSS منتقل می شود و مقدار ورودی پین NSS نادیده گرفته می شود.
۷	LSBFIRST	فرمت فریم
	۰	MSB (بیت مهم) اول ارسال می شود
	۱	LSB (بیت کم اهمیت) اول ارسال می شود
۶	SPE	فعال سازی SPI
۵:۳	BR[2:0]	کنترل نرخ Baud 000: fPCLK/32 001: fPCLK/64 010: fPCLK/128 011 fPCLK/256
۲	MSTR	انتخاب Master
۱	CPOL	قطبیت ساعت
	۰	CK: 0 در زمانی که بی کار است
	۱	CK: 1 در زمانی که بی کار است
۰	CPHA	فاز ساعت
	۰	اولین انتقال ساعت، لبه اول دریافت داده است
	۱	انتقال دوم ساعت، لبه اول دریافت داده است

SPI control register 2 (SPI_CR2) 15.3.2

To be defined

SPI status register (SPI_SR) 15.3.3

رجیستر **SPI Status Register (SPI_SR)** در میکروکنترلر STM32F407 وضعیت رابط SPI را نشان می‌دهد و شامل چندین فلگ است که به برنامه‌نویس اطلاعات مهمی در مورد وضعیت تبادل داده می‌دهد. این رجیستر از چندین بیت تشکیل شده است که هر یک نشان‌دهنده وضعیت خاصی هستند.

SPI status register (SPI_SR)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							FRE	BSY	OVR	MODF	CRC ERR	UDR	CHSIDE	TXE	RXNE
							r	r	r	r	rc_w0	r	r	r	r

توضیحات ۰: غیر فعال ۱: فعال	نام	بیت
به‌طور خودکار توسط سخت‌افزار به ۰ تنظیم می‌شوند.	رزرو	۱۵:۹
خطای فرمت فریم	FRE	۸
علامت مشغول بودن	BSY	۷
علامت سرریز	OVR	۶
خطای حالت	MODF	۵
علامت خطای CRC	CRCERR	۴
علامت کمبود داده	UDR	۳
طرف کانال	CHSIDE	۲
کانال چپ باید ارسال شود یا دریافت شده است	۰	
کانال راست باید ارسال شود یا دریافت شده است	۱	
بافر ارسال خالی	TXE	۱
بافر دریافت پر	RXNE	۰

SPI data register (SPI_DR) 15.3.4

این رجیستر برای انتقال داده بین میکروکنترلر و دستگاه‌های جانبی (مانند سنسورها، حافظه‌ها، و غیره) استفاده می‌شود. هر بار که داده‌ای به SPI_DR نوشته شود، این داده ارسال می‌شود و وقتی داده‌ای دریافت می‌شود، در همین رجیستر ذخیره می‌شود.

SPI data register (SPI_DR)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

SPI CRC polynomial register (SPI_CRCPR) (not used in I2S mode) 15.3.5

این رجیستر شامل چندجمله‌ای برای محاسبه CRC است.

SPI CRC polynomial register (SPI_CRCPR) (not used in I ² S mode)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRCPOLY[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

SPI RX CRC register (SPI_RXCRCR) (not used in I2S mode) 15.3.6

SPI RX CRC register (SPI_RXCRCR) (not used in I ² S mode)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RXCRC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

زمانی که محاسبه CRC فعال باشد، بیت‌های [15:0] RxCRC شامل مقدار محاسبه شده CRC برای بایت‌های دریافتی بعدی CRC هستند و به‌صورت سریالی با استفاده از چندجمله‌ای برنامه‌ریزی شده در رجیستر SPI_CRCPR محاسبه می‌شود.

15.3.7 SPI TX CRC register (SPI_TXCRCR) (not used in I²S mode)

SPI TX CRC register (SPI_TXCRCR) (not used in I ² S mode)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TXCRC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

زمانی که محاسبه CRC فعال باشد، بیت‌های [7:0] TxCRC شامل مقدار محاسبه شده CRC برای بایت‌های ارسالی بعدی هستند.

15.3.8 SPI_I²S configuration register (SPI_I2SCFGR)

SPI_I ² S configuration register (SPI_I2SCFGR)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				I2SMOD	I2SE	I2SCFG		PCMSY NC	Reserved	I2SSTD		CKPOL	DATLEN		CHLEN
				r/w	r/w	r/w	r/w	r/w		r/w	r/w	r/w	r/w	r/w	r/w
بیت				نام				توضیحات							
								۰: غیر فعال ۱: فعال							
۱۵:۱۲				رژرو				به‌طور خودکار توسط سخت‌افزار به ۰ تنظیم می‌شوند.							
۱۱				I2SMOD				انتخاب حالت I2S							
				۰				حالت SPI انتخاب شده است							
				۱				حالت I2S انتخاب شده است							
10				I2SE				فعال‌سازی I2S							
9:8				I2SCFG:				حالت پیکربندی I2S							
				00				Slave - ارسال							

	01	Slave دریافت
	10	Master ارسال
	11	Master دریافت
7	PCMSYNC	همزمانی فریم PCM
	0	همزمانی فریم کوتاه
	1	همزمانی فریم طولانی
6	رزرو	به طور خودکار توسط سخت افزار به ۰ تنظیم می شوند.
5:4	I2SSTD	انتخاب استاندارد I2S
	00	استاندارد فیلپس I2S
	01	استاندارد (MSB justified) چپ چین
	10	استاندارد (LSB justified) راست چین
	11	استاندارد PCM
3	CKPOL	قطبیت ساعت در حالت پایدار
	0	حالت پایدار ساعت I2S در سطح پایین است
	1	حالت پایدار ساعت I2S در سطح بالا است
2:1	DATLEN	طول داده ای که باید منتقل شود
	00	طول داده ۱۶ بیتی
	01	طول داده ۲۴ بیتی
	10	طول داده ۳۲ بیتی
	11	مجاز نیست
0	CHLEN	طول کانال (تعداد بیت ها در هر کانال صوتی)
	0	16 بیت عرض
	1	32 بیت عرض

15.4 پیکربندی های ساعت SPI

دستگاه مستر SPI مسئول تولید سیگنال ساعت برای آغاز و ادامه فرآیند انتقال داده است. بنابراین، مستر نرخ داده را با کنترل فرکانس خط ساعت سریال (SCK) که یک پارامتر قابل برنامه ریزی در سخت افزار است، تعیین می کند.

ساعت SPI دو پارامتر دیگر برای کنترل دارد که شامل فاز ساعت (CPHA) و قطبیت ساعت (CPOL) است. که ۴ مد کاری مانند شکل ۳-۱۵ برای SPI تعیین میکنند.

SPI mode	Clock polarity (CPOL)	Clock phase (CPHA)	Data is shifted out on	Data is sampled on
0	0	0	falling SCLK, and when $\overline{CS}$ activates	rising SCLK
1	0	1	rising SCLK	falling SCLK
2	1	0	rising SCLK, and when $\overline{CS}$ activates	falling SCLK
3	1	1	falling SCLK	rising SCLK

شکل ۳-۱۵: مدهای کاری SPI

15.5 فرمت فریم داده SPI STM32

داده‌ها می‌توانند به صورت MSB-first یا LSB-first منتقل شوند که بستگی به مقدار بیت LSBFIRST در رجیستر SPI_CR1 دارد.

هر فریم داده ۸ یا ۱۶ بیت طول دارد که بستگی به اندازه داده برنامه‌ریزی شده با استفاده از بیت DFF در رجیستر SPI_CR1 دارد. فرمت فریم داده انتخاب شده باید برای انتقال و/یا دریافت یکسان باشد.

اسلیو باید بداند که چه چیزی را انتظار داشته باشد تا بتواند داده‌های دریافتی را به درستی بخواند. این فرمت باید به‌طور هماهنگ با مستر تنظیم شود.

15.6 SPI STM32 در حالت اسلیو

در پیکربندی اسلیو، ساعت سریال از پین SCK از دستگاه مستر دریافت می‌شود. مقدار نرخ باود تنظیم شده در بیت‌های BR[۲:۰] در رجیستر SPI_CR1، بر نرخ انتقال داده تأثیر نمی‌گذارد. در این پیکربندی، پین MOSI یک پین ورودی داده و پین MISO یک پین خروجی داده است.

توجه داشته باشید که باید دستگاه اسلیوی SPI قبل از اینکه مستر شروع به ارسال کند، راه‌اندازی و پیکربندی شده باشد تا از هرگونه خراب شدن داده در ابتدای ارتباط جلوگیری شود.

15.6.1 توالی انتقال در حالت اسلیو

بایت داده به‌طور موازی در بافر Tx در طول یک چرخه نوشتن بارگذاری می‌شود. توالی انتقال زمانی آغاز می‌شود که دستگاه اسلیو سیگنال ساعت و بیت مهمترین داده را در پین MOSI خود دریافت کند. بیت‌های باقی‌مانده (۷ بیت در فرمت فریم ۸ بیتی و ۱۵ بیت در فرمت فریم ۱۶ بیتی) در شیفت رجیستر بارگذاری می‌شوند. پرچم TXE در رجیستر SPI_SR در زمان انتقال داده از بافر Tx به شیفت رجیستر تنظیم می‌شود و یک وقفه ایجاد می‌شود اگر بیت TXEIE در رجیستر SPI_CR2 تنظیم شده باشد.

15.6.2 توالی دریافت در حالت اسلیو

برای دریافت‌کننده، زمانی که انتقال داده کامل می‌شود: داده در شیفت رجیستر به بافر Rx منتقل می‌شود و پرچم RXNE (رجیستر SPI_SR) تنظیم می‌شود و اگر بیت RXNEIE در رجیستر SPI_CR2 تنظیم شده باشد یک وقفه ایجاد می‌شود. بیت RXNE پس از آخرین لبه ساعت نمونه‌برداری، تنظیم می‌شود و یک نسخه از بایت داده دریافتی در شیفت رجیستر به بافر Rx منتقل می‌شود. وقتی رجیستر SPI_DR خوانده می‌شود، عملاً مقدار بافر شده خوانده می‌شود. پاک‌سازی بیت RXNE با خواندن رجیستر SPI_DR انجام می‌شود.

15.7 SPI STM32 در حالت مستر

در پیکربندی مستر، ساعت سریال بر روی پین SCK تولید می‌شود. در این پیکربندی، پین MOSI یک پین خروجی داده و پین MISO یک پین ورودی داده است.

15.7.1 توالی انتقال در حالت مستر

توالی انتقال زمانی آغاز می‌شود که یک بایت در بافر Tx نوشته می‌شود. بایت داده به‌طور موازی در شیفت رجیستر (از باس داخلی) در طول انتقال بیت اول بارگذاری می‌شود و سپس به‌صورت سریالی به پین MOSI منتقل می‌شود، چه به‌صورت MSB-first و چه LSB-first که بستگی به بیت LSBFIRST در رجیستر SPI_CR1 دارد.

پرچم TXE در زمان انتقال داده از بافر Tx به شیفت رجیستر تنظیم می‌شود و اگر بیت TXEIE در رجیستر SPI_CR2 تنظیم شده باشد، یک وقفه ایجاد می‌شود.

15.7.2 توالی دریافت در حالت مستر

برای دریافت‌کننده، زمانی که انتقال داده کامل می‌شود: داده در شیفت رجیستر به بافر RX منتقل می‌شود و پرچم RXNE تنظیم می‌شود. یک وقفه ایجاد می‌شود اگر بیت RXNEIE در رجیستر SPI_CR2 تنظیم شده باشد.

در آخرین لبه ساعت نمونه‌برداری، بیت RXNE تنظیم می‌شود و یک نسخه از بایت داده دریافتی در شیفت رجیستر به بافر Rx منتقل می‌شود. وقتی رجیستر SPI_DR خوانده می‌شود، سخت‌افزار SPI این مقدار بافر شده را بازمی‌گرداند.

پاک‌سازی بیت RXNE با خواندن رجیستر SPI_DR انجام می‌شود. یک جریان انتقال مداوم می‌تواند حفظ شود اگر داده بعدی که قرار است ارسال شود، به محض شروع انتقال در بافر Tx قرار گیرد. توجه داشته باشید که پرچم TXE قبل از هر تلاشی برای نوشتن در بافر Tx باید '۱' باشد.

15.8 پرچم‌های SPI STM32 (وضعیت و خطاها)

پرچم‌های وضعیتی برای برنامه‌نویسی فراهم شده‌اند تا وضعیت باس SPI به‌طور کامل تحت نظر قرار گیرد. وضعیت SPI را در رجیستر SPI_SR قابل مشاهده است.

- پرچم خالی بودن بافر Tx (TXE) - زمانی که این پرچم تنظیم می‌شود، نشان‌دهنده این است که بافر Tx خالی است و داده بعدی برای انتقال می‌تواند در بافر بارگذاری شود. پرچم TXE هنگام نوشتن در رجیستر SPI_DR پاک می‌شود.

- پرچم پر بودن بافر Rx (RXNE) - زمانی که تنظیم می‌شود، نشان‌دهنده این است که داده‌های دریافتی معتبری در بافر Rx وجود دارد. این پرچم هنگام خواندن SPI_DR پاک می‌شود.

- پرچم مشغول بودن (BUSY) - پرچم BSY برای شناسایی پایان یک انتقال مفید است اگر نرم‌افزار بخواهد SPI را غیرفعال کرده و به حالت Halt برود (یا ساعت پیرامون را غیرفعال کند). این از فساد آخرین انتقال جلوگیری می‌کند. برای این کار، رویه زیر باید به‌دقت رعایت شود. پرچم BSY همچنین برای جلوگیری از تصادف‌های نوشتن در یک سیستم چند مستر مفید است.

پرچم‌های دیگری نیز وجود دارند که نشان می‌دهند آیا نوع خاصی از خطا رخ داده است یا خیر.

- خطای حالت مستر SPI (MODF) - خطای حالت مستر زمانی رخ می‌دهد که دستگاه مستر پین NSS خود را به پایین کشیده باشد (در حالت سخت‌افزاری NSS) یا بیت SSI را پایین نگه‌داشته باشد (در حالت نرم‌افزاری NSS)، که به‌طور خودکار بیت MODF را تنظیم می‌کند.

- شرایط Overrun در SPI - شرایط Overrun زمانی رخ می‌دهد که دستگاه مستر بایت‌های داده را ارسال کرده باشد و دستگاه اسلیو بیت RXNE را که ناشی از بایت داده قبلی منتقل شده، پاک نکرده باشد.

- خطای CRC در SPI - این پرچم برای تأیید صحت مقدار دریافتی زمانی استفاده می‌شود که بیت CRCEN در رجیستر SPI_CR1 تنظیم شده باشد. پرچم CRCERR در رجیستر SPI_SR تنظیم می‌شود اگر مقدار دریافتی در شیفت رجیستر با مقدار SPI_RXCRCR دریافت‌کننده مطابقت نداشته باشد.

15.9 وقفه‌های SPI STM32

رویدادهای وقفه SPI به یک وکتور وقفه مشترک متصل هستند. به این ترتیب، SPI یک سیگنال وقفه واحد را بدون توجه به منبع آن تولید می‌کند. نرم‌افزار باید این سیگنال را شناسایی کند. این رویدادها تنها در صورتی یک وقفه ایجاد می‌کنند که بیت کنترل فعال مربوطه تنظیم شده باشد.

SPI interrupt requests

Interrupt event	Event flag	Enable Control bit
Transmit buffer empty flag	TXE	TXEIE
Receive buffer not empty flag	RXNE	RXNEIE
Master Mode fault event	MODF	ERRIE
Overrun error	OVR	
CRC error flag	CRCERR	
TI frame format error	FRE	ERRIE

در این بخش، روش‌های ممکن برای مدیریت تراکنش‌های SPI در برنامه‌های نرم‌افزاری شما را فهرست می‌کنم. برای مثال‌های کد و آزمایش‌ها، فقط روی دکمه آموزش بعدی کلیک کنید و به مرور این سری آموزش‌ها ادامه دهید. مثال‌ها و کتابخانه‌های زیادی بر اساس ارتباط SPI خواهیم ساخت.

15.10.1 SPI با بررسی (Polling)

اولین و ساده‌ترین راه برای انجام هر کاری در نرم‌افزارهای توکار، بررسی منابع سخت‌افزاری تا زمانی است که آماده حرکت به مرحله بعدی در دستورالعمل‌های برنامه شما باشد. با این حال، این روش کمترین کارایی را دارد و CPU زمان زیادی را در حالت "مشغول-منتظر" busy-waiting هدر می‌دهد.

این موضوع هم برای انتقال و هم برای دریافت صدق می‌کند. شما فقط منتظر می‌مانید تا بایت فعلی داده برای انتقال آماده شود تا بتوانید بایت بعدی را شروع کنید و به همین ترتیب ادامه دهید.

15.10.2 SPI با وقفه‌ها (Interrupts)

با این حال، می‌توانیم وقفه‌های SPI را فعال کنیم و سیگنالی دریافت کنیم زمانی که کار انجام شده و آماده سرویس‌دهی به CPU است. چه برای داده‌هایی که ارسال شده‌اند و چه برای داده‌هایی که دریافت شده‌اند. این روش زمان زیادی را صرفه‌جویی می‌کند و همیشه بهترین راه برای مدیریت چنین رویدادهایی بوده است.

با این حال، در برخی از برنامه‌های "زمان بحرانی"، نیاز داریم که همه چیز به‌طور تعیین‌شده و در زمان مشخصی انجام شود. یک مشکل عمده با وقفه‌ها این است که نمی‌توانیم پیش‌بینی کنیم که چه زمانی به وقوع می‌پیوندد یا در کدام وظیفه. این می‌تواند رفتار زمان‌بندی سیستم را مختل کند، به‌ویژه با یک باس ارتباطی بسیار سریع مانند SPI که می‌تواند CPU را در صورت دریافت بلوک‌های داده بزرگ با نرخ بسیار بالا مسدود کند.

15.10.3 DMA با SPI

برای عملکرد در حداکثر سرعت، SPI نیاز دارد که با داده‌های برای انتقال تغذیه شود و داده‌های دریافتی در بافر Rx باید خوانده شوند تا از وقوع Overrun جلوگیری شود. برای تسهیل انتقال، SPI قابلیت DMA را ارائه می‌دهد که یک پروتکل ساده درخواست/تأیید را پیاده‌سازی می‌کند.

دسترسی به DMA زمانی درخواست می‌شود که بیت فعال‌سازی در رجیستر SPI_CR2 فعال باشد. درخواست‌های جداگانه‌ای باید برای بافرهای Tx و Rx صادر شوند.

استفاده از واحد DMA نه تنها باعث می‌شود که SPI با حداکثر سرعت در هر دو سمت ارتباط عمل کند، بلکه CPU را از انجام انتقال داده‌ها "از سخت‌افزار به حافظه" آزاد می‌کند. این در نهایت زمان زیادی را صرفه‌جویی می‌کند و به‌عنوان کارآمدترین روش برای مدیریت این انتقال داده از سخت‌افزار به حافظه و بالعکس در نظر گرفته می‌شود.

15.11 توابع HAL در حالت‌های مختلف SPI STM32

15.11.1 1. حالت Blocking برای انتقال

-انتقال :

```
HAL_SPI_Transmit(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout);
```

- دریافت :

```
HAL_SPI_Receive(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout);
```

- انتقال-دریافت :

```
HAL_SPI_TransmitReceive(SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size, uint32_t Timeout);
```

15.11.2 2. حالت Interrupt برای انتقال

-انتقال :

```
HAL_SPI_Transmit_IT(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size);
```

پس از فراخوانی این تابع، سختافزار SPI شروع به ارسال همه بایت‌های داده در بافر یکی یکی می‌کند تا تمام شود. وقتی تمام شد، تابع زیر فراخوانی و اجرا می‌شود. اگر می‌خواهید کاری در هنگام اتمام انتقال داده انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // TX Done .. Do Something ...
}
```

-دریافت :

```
HAL_SPI_Receive_IT(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size);
```

پس از فراخوانی این تابع، سختافزار SPI شروع به دریافت همه بایت‌های داده ورودی در بافر یکی یکی می‌کند تا تمام شود. وقتی تمام شد، تابع زیر فراخوانی و اجرا می‌شود. اگر می‌خواهید کاری در هنگام اتمام دریافت داده انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

```
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // RX Done .. Do Something ...
}
```

-انتقال-دریافت :

```
HAL_SPI_TransmitReceive_IT(SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t *
pRxData, uint16_t Size);
```


پس از فراخوانی این تابع، سختافزار SPI شروع به ارسال و دریافت همه بایت‌های داده ورودی در بافرها یکی یکی می‌کند تا تمام شود. وقتی تمام شد، تابع زیر فراخوانی و اجرا می‌شود. اگر می‌خواهید کاری در هنگام اتمام انتقال-دریافت انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // TX-RX Done .. Do Something ...
}
```

15.11.3 3. حالت DMA برای انتقال

-انتقال :

```
HAL_SPI_Transmit_DMA(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size);
```

پس از فراخوانی این تابع، سختافزار SPI شروع به ارسال همه بایت‌های داده در بافر یکی یکی می‌کند تا تمام شود (در حالت DMA) وقتی تمام شد، تابع زیر فراخوانی و اجرا می‌شود. اگر می‌خواهید کاری در هنگام اتمام انتقال داده انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // TX Done .. Do Something ...
}
```

- دریافت :

```
HAL_SPI_Receive_DMA(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size);
```

پس از فراخوانی این تابع، سخت‌افزار SPI شروع به دریافت همه بایت‌های داده ورودی در بافر یکی یکی می‌کند تا تمام شود) در حالت (DMA وقتی تمام شد، تابع زیر فراخوانی و اجرا می‌شود. اگر می‌خواهید کاری در هنگام اتمام دریافت داده انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

```
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // RX Done .. Do Something ...
}
```

- انتقال-دریافت :

```
HAL_SPI_TransmitReceive_DMA(SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t *
pRxData, uint16_t Size);
```

پس از فراخوانی این تابع، سخت‌افزار SPI شروع به ارسال و دریافت همه بایت‌های داده ورودی در بافرها یکی یکی می‌کند تا تمام شود) در حالت (DMA وقتی تمام شد، تابع زیر فراخوانی و اجرا می‌شود. اگر می‌خواهید کاری در هنگام اتمام انتقال-دریافت انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // TX-RX Done .. Do Something ...
}
```

15.12 پروژه انتقال داده با SPI

در این آموزش، ما به نحوه ارسال و دریافت داده‌های SPI با میکروکنترلرهای STM32 در حالت‌های DMA ، وقفه و بررسی (Polling) خواهیم پرداخت. ابتدا با پروژه نرم‌افزاری SPI Master (فرستنده) شروع می‌کنیم و سپس تنظیمات آزمایش (برد SPI Master -> SPI Slave) را برای آزمایش‌هایی که در ادامه انجام خواهیم داد، نشان می‌دهم.

پس از پیکربندی دستگاه SPI در حالت Master، چه در CubeMX و چه از طریق دسترسی به رجیسترها، می‌توانیم انتقال داده به دستگاه‌های SPI Slave را آغاز کنیم. چندین روش مختلف برای انجام انتقال داده از طریق باس SPI وجود دارد و در اینجا به برخی از آن‌ها اشاره می‌کنیم.

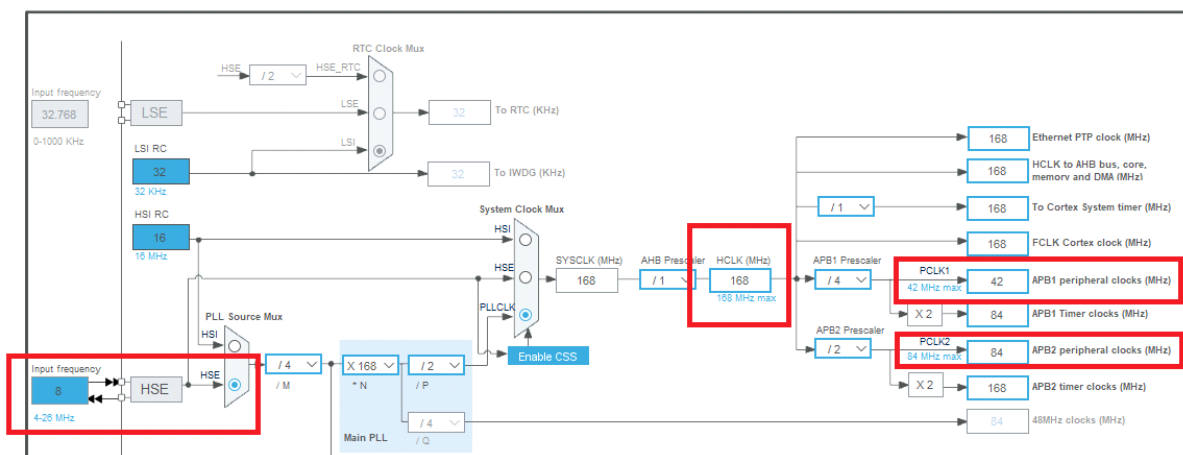
15.12.1 انتقال داده با SPI به روش Polling

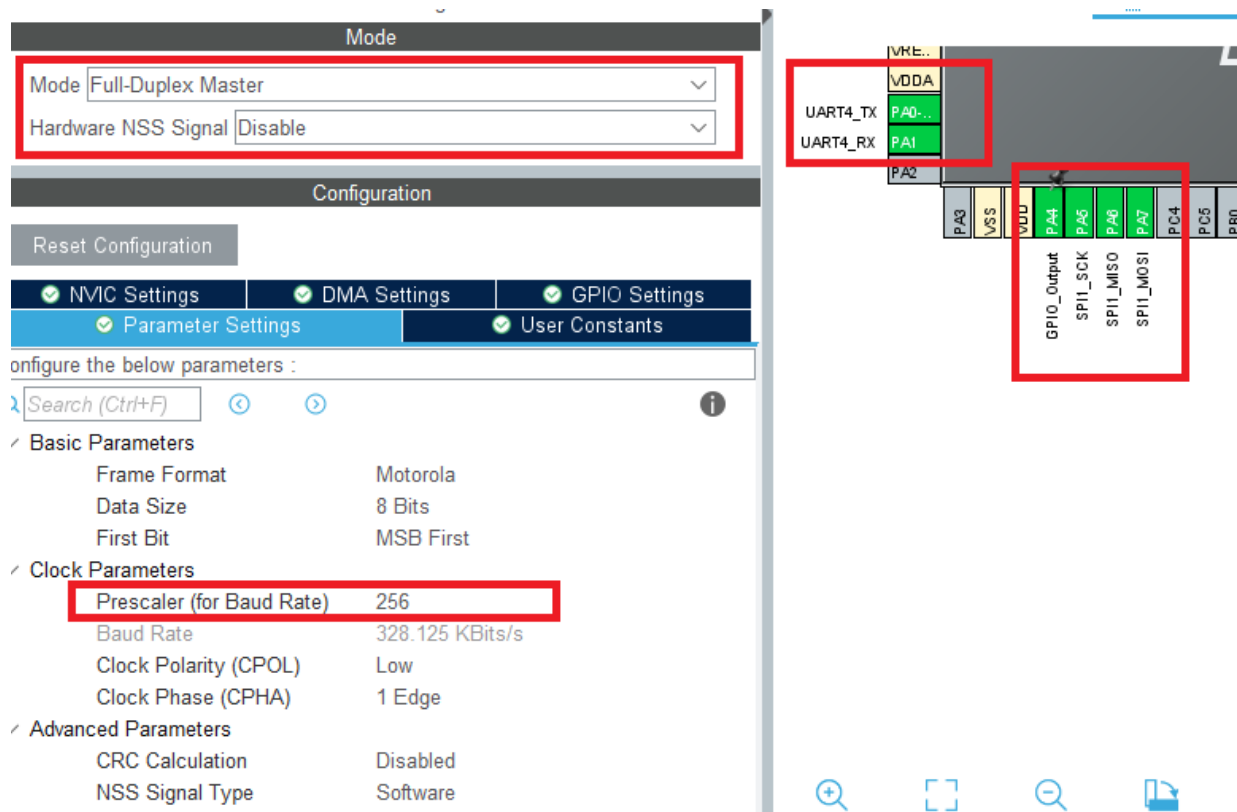
اولین روش برای ارسال یک بافر داده از طریق باس SPI، استفاده از روش بررسی (Polling) است که یک قطعه کد مسدودکننده است و منتظر می‌ماند تا بایت فعلی به طور کامل منتقل شود، سپس بایت بعدی را ارسال می‌کند و به همین ترتیب ادامه می‌دهد.

این روش ساده‌ترین راه برای پیاده‌سازی است، اما زمان‌برترین روش برای CPU خواهد بود که در نهایت برای مدتی به حالت "انتظار مشغول" می‌رسد که غیرضروری است.

در این پروژه مستر دادهایی را برای slave می‌فرستد و slave پس از دریافت روی uart نمایش می‌دهد. در این پروژه به دو باکس نیاز هست که یکی مستر و دیگری slave خواهد بود. لذا تنظیمات هر دو به طور جداگانه ارائه شده است.

پیکربندی Master





کدهای را در محیط keil مطابق کد ذیل ویرایش نمایید.

Master Code
<pre>#include "main.h" #define DATA_SIZE 10 uint8_t txData[DATA_SIZE] = {"1122334455"}; uint8_t rxData[DATA_SIZE]; SPI_HandleTypeDef hspi1; UART_HandleTypeDef huart4; void SystemClock_Config(void); static void MX_GPIO_Init(void); static void MX_SPI1_Init(void);</pre>

```

static void MX_UART4_Init(void);

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_UART4_Init();

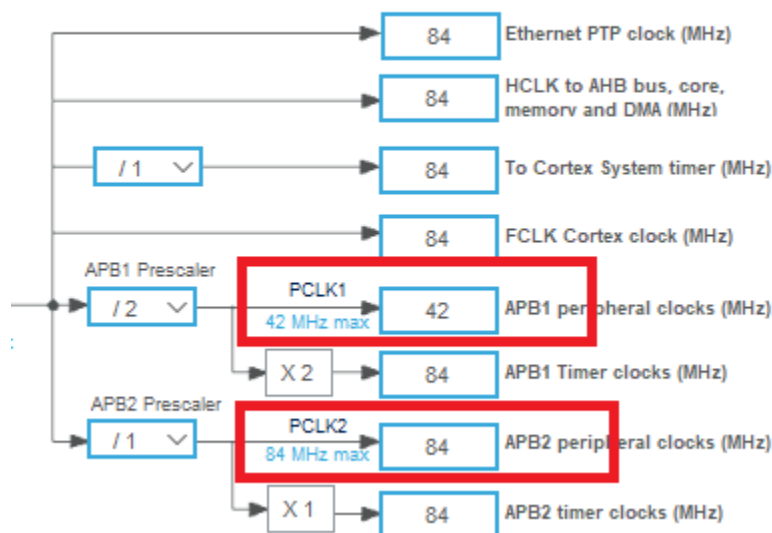
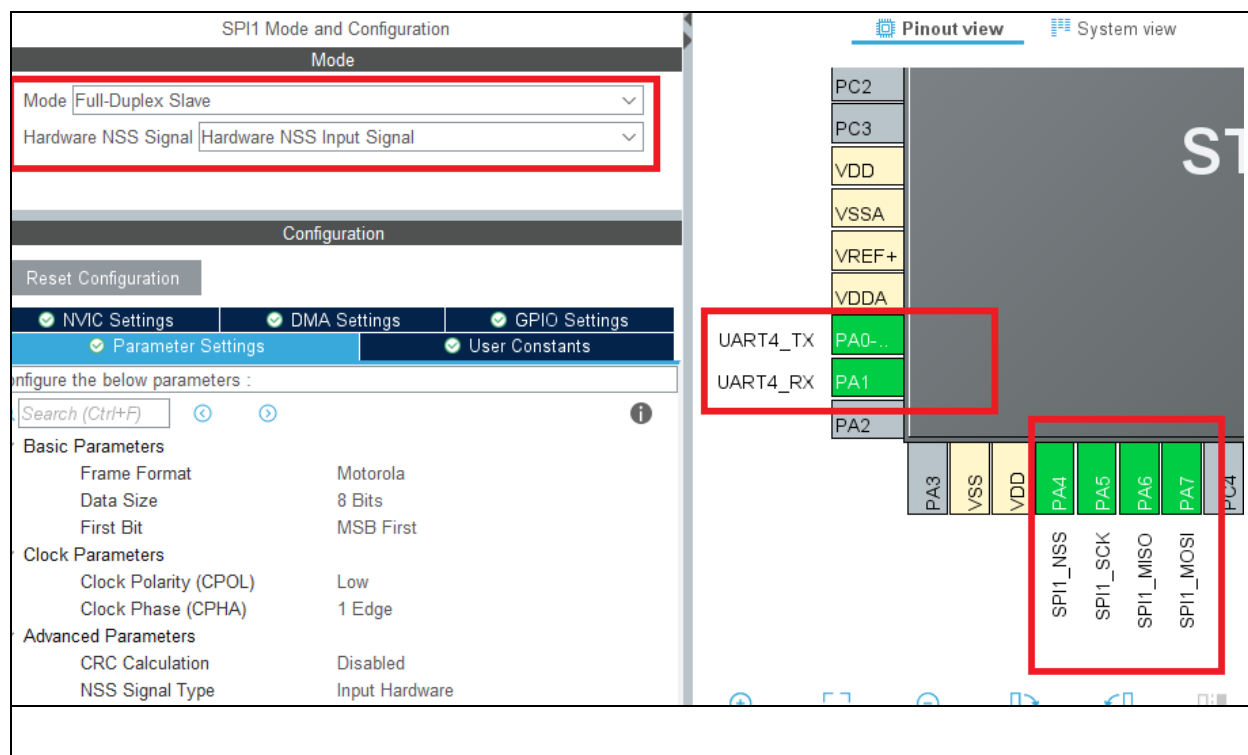
    HAL_UART_Transmit(&huart4,"microlab master\r\n",sizeof("microlab master\r\n"), HAL_MAX_DELAY);
    while (1)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);

        HAL_SPI_TransmitReceive(&hspi1, txData, rxData, DATA_SIZE, HAL_MAX_DELAY);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);

        HAL_UART_Transmit(&huart4, rxData, DATA_SIZE, HAL_MAX_DELAY);
        HAL_Delay(1000);
    }
}

```

پی‌کربندی Slave



Slave Code

```
#include "main.h"

#define DATA_SIZE 10

uint8_t txData[DATA_SIZE] = { 's', 'a', 'l', 'a', 'm', '1', '2', '3', '\r', '\n' };
//uint8_t txData[DATA_SIZE] = {"salam123\r\n"};
uint8_t rxData[DATA_SIZE];

SPI_HandleTypeDef hspi1;

UART_HandleTypeDef huart4;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);
static void MX_UART4_Init(void);

int main(void)
{
    HAL_Init();

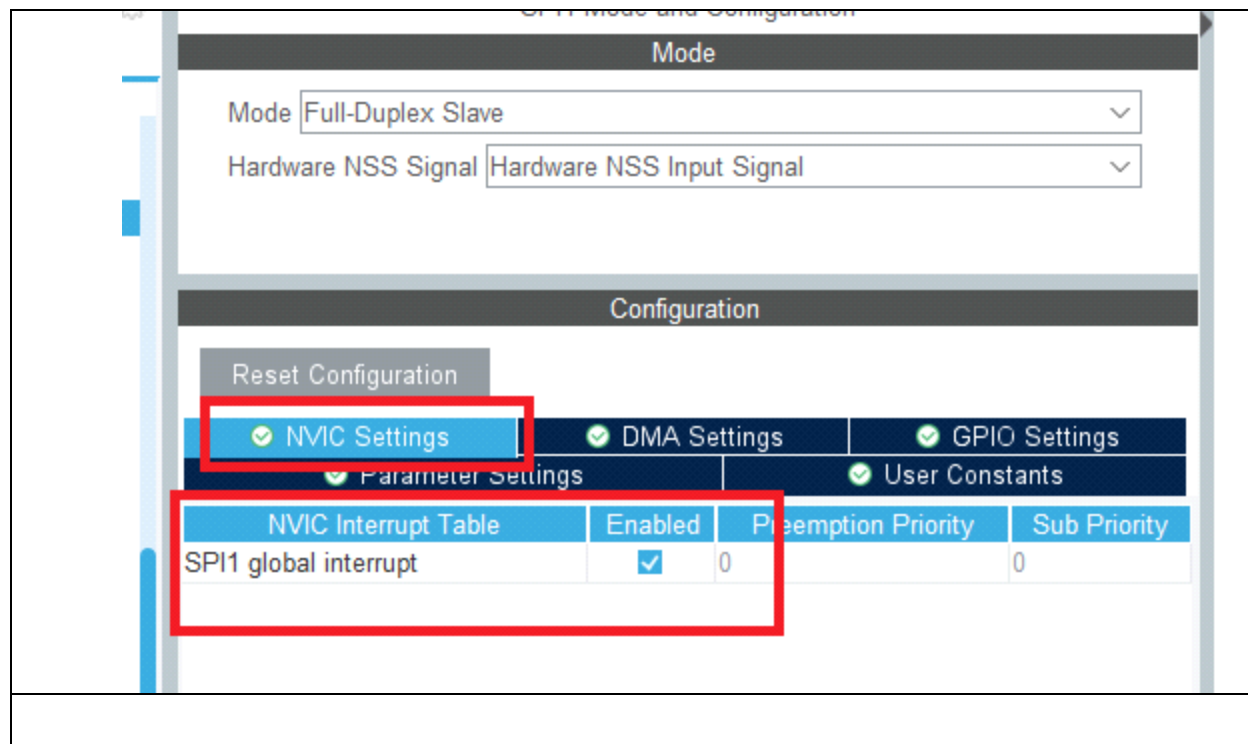
    SystemClock_Config();

    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_UART4_Init();

    HAL_UART_Transmit(&huart4,"microlab slave\r\n",sizeof("microlab slave\r\n"), HAL_MAX_DELAY);
    while (1)
    {
        HAL_SPI_TransmitReceive(&hspi1, txData, rxData, DATA_SIZE, HAL_MAX_DELAY);
        HAL_UART_Transmit(&huart4,"\r\n your data is:",sizeof("\r\n your data is:"), HAL_MAX_DELAY);
        HAL_UART_Transmit(&huart4,rxData,DATA_SIZE, HAL_MAX_DELAY);
        HAL_Delay(1000);
    }
}
```

15.12.2 انتقال داده با روش وقفه در SPI

در این روش مستر مانند روش قبلی به روش polling داده میفرستد و اسلیو با وقفه داده‌ها را دریافت مینماید. تنظیمات اسلیو در شکل نشان داده شده است.



کدهای ایجاد شده را در keil مشابه کد ذیل اصلاح نمایید.

Slave Code
<pre>#include "main.h" #define DATA_SIZE 10 uint8_t txData[DATA_SIZE] = { 's', 'a', 'l', 'a', 'm', '1', '2', '3', '\r', '\n' }; //uint8_t txData[DATA_SIZE] = {"salam123\r\n"}; uint8_t rxData[DATA_SIZE]; SPI_HandleTypeDef hspi1; UART_HandleTypeDef huart4;</pre>


```

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);
static void MX_UART4_Init(void);

void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi){
    if (hspi->Instance == SPI1)
    {
        HAL_UART_Transmit(&huart4, (uint8_t *)"\r\n your data is:", sizeof("\r\n your data is:"),
        HAL_MAX_DELAY);
        HAL_UART_Transmit(&huart4, rxData, DATA_SIZE, HAL_MAX_DELAY);
        HAL_SPI_TransmitReceive_IT(&hspi1, txData, rxData, DATA_SIZE);
    }
}

int main(void)
{
    HAL_Init();

    SystemClock_Config();

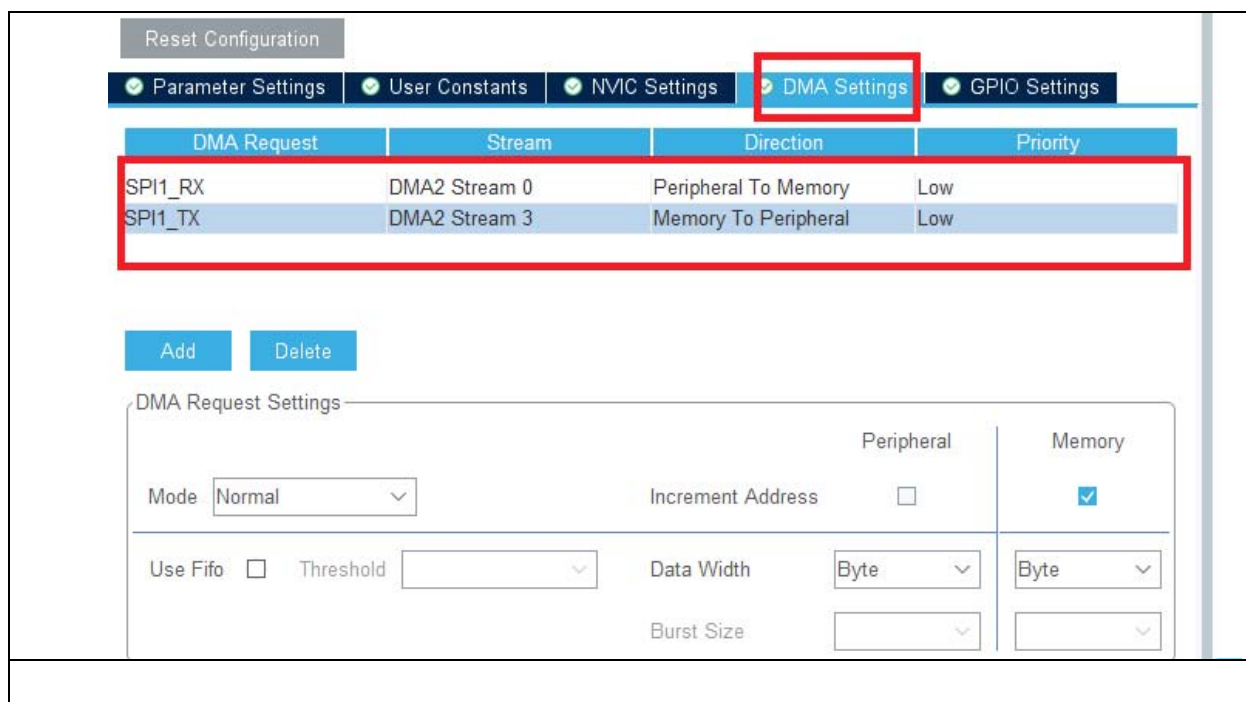
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_UART4_Init();

    HAL_UART_Transmit(&huart4,"microlab slave\r\n",sizeof("microlab slave\r\n"),
    HAL_MAX_DELAY);
    HAL_SPI_TransmitReceive_IT(&hspi1, txData, rxData, DATA_SIZE);
    while (1)
    {
        HAL_Delay(1000);
    }
}

```

5.12.3 انتقال داده با SPI به روش DMA

در این روش هم مستر با روش polling ارسال نموده و اسلیو با روش DMA داده ها را دریافت مینماید.



در محیط keil فقط کافی است در برنامه وقفه دستورات HAL_SPI_TransmitReceive_IT را به HAL_SPI_TransmitReceive_DMA تغییر دهید.

15.12.4 ارتباط با ADC خارجی از طریق SPI

در پکیج آزمایشگاه تراشه ADC به نام MCP3202 وجود دارد. مبدل آنالوگ به دیجیتال MCP3202 یک IC بسیار محبوب از شرکت Microchip است که برای تبدیل سیگنال‌های آنالوگ به دیجیتال استفاده می‌شود. این مبدل دارای ویژگی‌های زیر است:

۱. رزولوشن: MCP3202 دارای رزولوشن ۱۲ بیتی است، که می‌تواند ۴۰۹۶ سطح مختلف را برای سیگنال آنالوگ شناسایی کند.

۲. کانال‌ها: این مبدل قابلیت خواندن از دو کانال ورودی آنالوگ را دارد (کانال‌های متفاوت A و B).

۳. رابط MCP3202 SPI از رابط SPI (Serial Peripheral Interface) برای ارتباط با میکروکنترلرها استفاده می‌کند، که امکان سرعت بالا و سادگی در ارتباط را فراهم می‌کند.

۴. سرعت تبدیل: سرعت تبدیل این دستگاه معمولاً در حدود ۱۰۰ کیلوهرتز است.

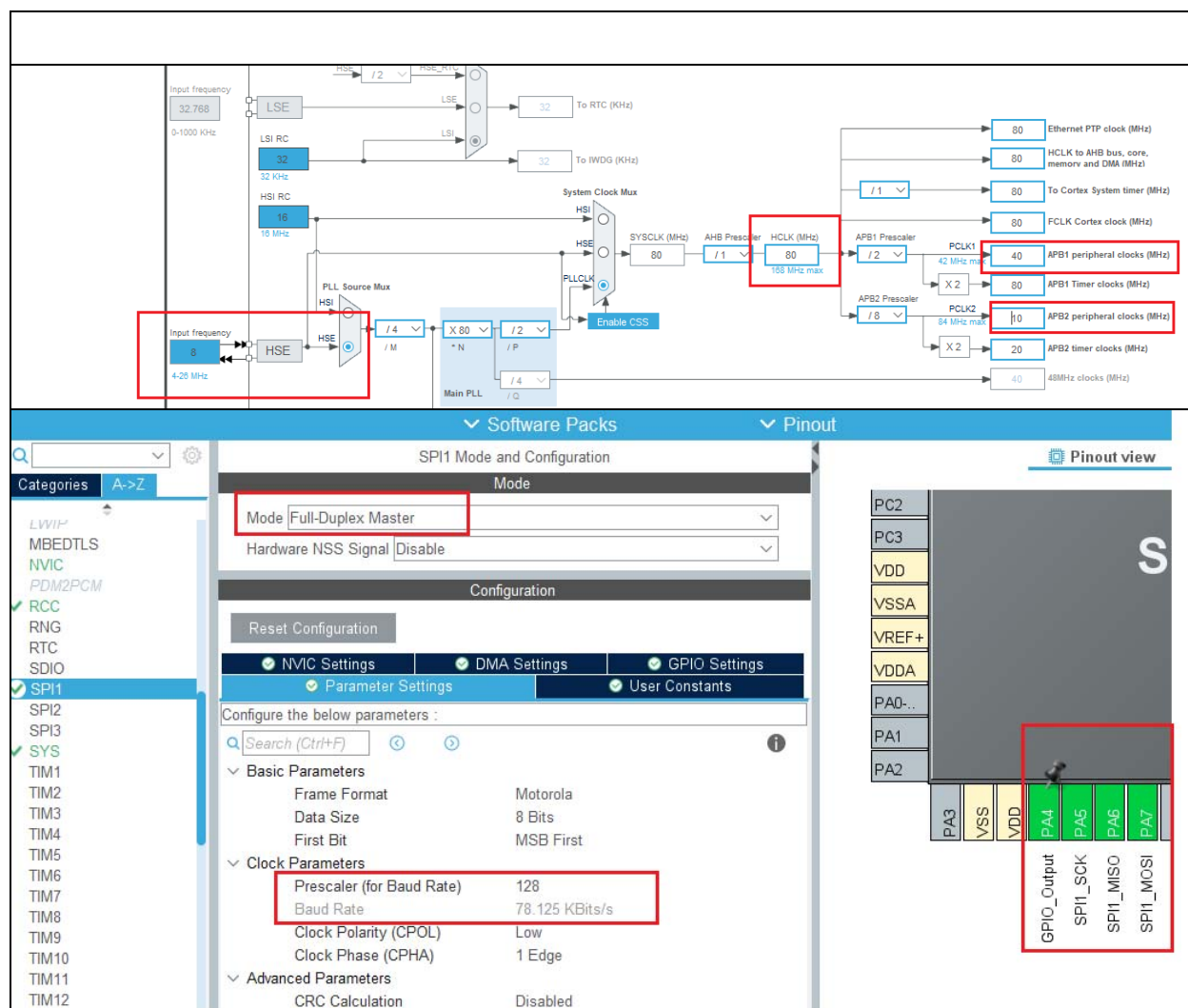
۵. ولتاژ تغذیه: این مبدل می‌تواند با ولتاژ تغذیه ۷۲.۷ تا ۷۵.۵ کار کند.

۶. کاربردها: MCP3202 در پروژه‌های مختلف الکترونیکی، از جمله اندازه‌گیری دما، فشار، و سایر سنسورهای آنالوگ مورد استفاده قرار می‌گیرد.

ابتدا باید اتصالات مبدل با میکروکنترلر STM32F407 را انجام دهید. اتصالات اصلی به این شکل است:

MCP3202 Pin	STM32F407 Pin	توضیحات
VDD	3.3V	تغذیه مبدل
VREF	3.3V	ولتاژ مرجع (بر اساس نیاز)
GND	GND	زمین
CLK	PA5	SPI (SCK) ساعت
DIN	PA7	SPI (MOSI) ورودی داده
DOUT	PA6	SPI (MISO) خروجی داده
CS	PA4	(Chip Select) انتخاب مدار

تنظیمات لازم در stm32cube را مشابه ذیل انجام دهید. uart را هم پیکربندی نمایید.



لازم به ذکر است که پایه PA4 به عنوان خروجی تعریف شده تا توسط نرم افزار مقدار مناسب برای CS تراشه ADC را به خود بگیرد.

کدها را در محیط keil مشابه ذیل اصلاح نمایید.

```
#include "main.h"
#include "stdio.h"

SPI_HandleTypeDef hspi1;
UART_HandleTypeDef huart2;

void SystemClock_Config(void);
```

```

static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);
static void MX_USART2_UART_Init(void);

int main(void)
{

    unsigned char txdata=0;
    unsigned char rxdata=0;
    char scr[32];

    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_USART2_UART_Init();

    while (1)
    {

        HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4, GPIO_PIN_RESET);
        txdata=0x01;
        HAL_SPI_TransmitReceive(&hspi1, &txdata, &rxdata, 1,200);
        txdata=0xA0;

        HAL_SPI_TransmitReceive(&hspi1, &txdata, &rxdata, 1,200);
        unsigned int adc_data0=(rxdata & 0x0f)<<8;
        txdata=0x00;
        HAL_SPI_TransmitReceive(&hspi1, &txdata, &rxdata, 1,200);
        adc_data0 |= rxdata;
        HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4, GPIO_PIN_SET);
        HAL_Delay(200);
        /*****/
        HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4, GPIO_PIN_RESET);
        txdata=0x01;
        HAL_SPI_TransmitReceive(&hspi1, &txdata, &rxdata, 1,200);
        txdata=0xE0;

        HAL_SPI_TransmitReceive(&hspi1, &txdata, &rxdata, 1,200);
        unsigned int adc_data1=(rxdata & 0x0f)<<8;
        txdata=0x00;
        HAL_SPI_TransmitReceive(&hspi1, &txdata, &rxdata, 1,200);
        adc_data1 |= rxdata;
        HAL_GPIO_WritePin(GPIOA,GPIO_PIN_4, GPIO_PIN_SET);

        sprintf(scr,"ADC0=%d--ADC1=%d\n\r",adc_data0,adc_data1);
        HAL_UART_Transmit(&huart2, scr, strlen(scr),HAL_MAX_DELAY);

        HAL_Delay(2000);
    }
}

```

}
}

15.12.5 ارتباط با DAC خارجی از طریق SPI

در پکیج آزمایشگاه تراشه MCP4921 وجود دارد. MCP4921 یک مبدل دیجیتال به آنالوگ 12 (DAC) بیتی از خانواده MCP می باشد که به راحتی می تواند در پروژه ها و طراحی های مختلف مورد استفاده قرار گیرد. در ادامه ویژگی ها، عملکرد و نکات مرتبط با آن آورده شده است.

ویژگی های: MCP4921

رزولوشن: MCP4921 دارای دقت ۱۲ بیتی است که امکان تولید ۴۰۹۶ سطح آنالوگ مختلف را فراهم می کند.

پروتکل ارتباطی: این DAC از پروتکل SPI برای ارتباط با میکروکنترلرها استفاده می کند، که آن را برای بسیاری از میکروکنترلرها (مانند STM32، AVR و PIC مناسب می سازد).

ولتاژ مرجع: می توان برای عملکرد DAC از ولتاژ مرجع داخلی (۲.۵ ولت) یا خارجی استفاده کرد. ولتاژ مرجع به دقت خروجی آنالوگ وابسته است.

خروجی آنالوگ: خروجی آنالوگ خروجی سه گانه ای است که در یک حالت بدون بار (انگلیسی buffered) و به صورت مستقیم برای بارهای الکتریکی قابل استفاده است.

مدل های مختلف: MCP4921 و MCP4922 (یک مدل با دو کانال) معمولاً استفاده می شوند که قابلیت های مشابهی دارند اما MCP4921 فقط یک کانال را ارائه می دهد.

کاربردها

- تولید سیگنال های آنالوگ : برای تولید سیگنال های آنالوگ در سیستم های صوتی و مخابراتی.
- کنترل موتور : برای کنترل سرعت و موقعیت موتورهای DC و سروو.
- سیستم های اندازه گیری : در سیستم های اندازه گیری و سنجش برای تولید ولتاژهای مرجع.
- سیستم های کنترلی : در سیستم های اتوماسیون صنعتی و کنترل فرآیند.

مزایا

- سادگی در استفاده : با استفاده از پروتکل SPI، ارتباط با میکروکنترلرها آسان است.
 - اندازه کوچک : این IC در بسته‌بندی‌های کوچک و متنوع موجود است که به صرفه‌جویی در فضا کمک می‌کند.
 - عملکرد پایدار : دقت و ثبات بالای خروجی ولتاژ.
- ولتاژ خروجی ایده‌آل با استفاده از معادله ۴-۱ محاسبه می‌شود، که در آن G نمایانگر بهره انتخاب‌شده (x_1 یا x_2)، DN نمایانگر مقدار ورودی دیجیتال و n تعداد بیت‌های دقت است.

$$V_{OUT} = \frac{V_{REF}GD_N}{2^n}$$

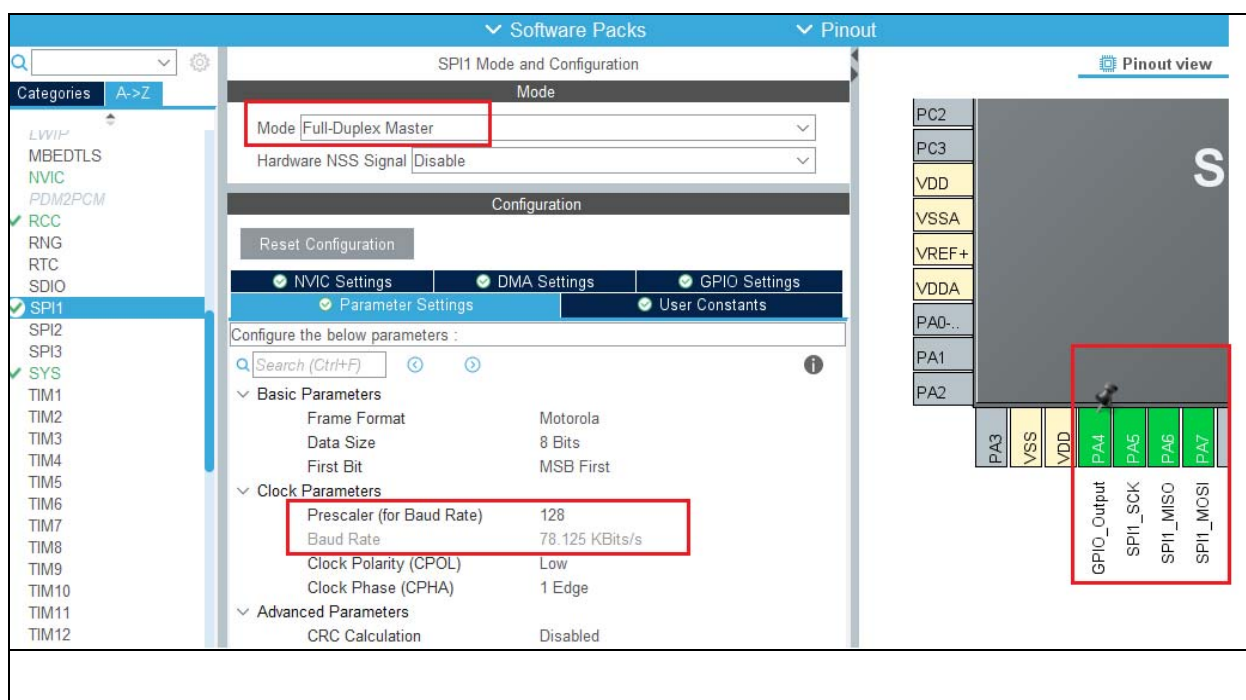
خانواده MCP492X به گونه‌ای طراحی شده است که به‌طور مستقیم با پورت رابط جانبی سری (SPI) که در بسیاری از میکروکنترلرها موجود است، ارتباط برقرار کند. در این حالت ارتباطات یک‌طرفه است و بنابراین، داده‌ها نمی‌توانند از MCP492X خوانده شوند. فرمان نوشتن شامل ۱۶ بیت است و برای پیکربندی و داده DAC استفاده می‌شود. در جدول مشخصات رجیستر ای تراشه نشان داده شده است.

WRITE COMMAND REGISTER							
Upper Half:							
W-x	W-x	W-x	W-0	W-x	W-x	W-x	W-x
$\overline{A/B}$	BUF	$\overline{GA}$	$\overline{SHDN}$	D11	D10	D9	D8
bit 15				bit 8			
Lower Half:							
W-x	W-x	W-x	W-x	W-x	W-x	W-x	W-x
D7	D6	D5	D4	D3	D2	D1	D0
bit 7				bit 0			

توضیحات	نام	شماره بیت
بیت انتخاب DACB یا DACB	A/B	۱۵
نوشتن به DACB	۰	
نوشتن به DACB	۱	
بیت کنترل بافر ورودی VREF	BUF	۱۴
بدون بافر	۰	
بافر شده	۱	
بیت انتخاب بهره خروجی	GA	۱۳
گین برابر با ۲	۰	
گین برابر با ۱	۱	

۱۲	SHDN	بیت کنترل خاموشی خروجی
	۰	بافر خروجی غیرفعال، خروجی در حالت امپدانس بالا
	۱	کنترل خاموشی خروجی
۱۱:۰	D11:D0	بیت‌های داده DAC شامل مقداری بین ۰ تا ۴۰۹۵ است.

تنظیمات لازم برای میکرو در محیط STM32cube را مشابه شکل انجام دهید. پایه PA4 به عنوان خروجی تعریف نمایید تا مقدار لازم برای CS تراشه DAC اتخاذ نماید.



اتصالات بین میکرو و DAC به شرح ذیل است:

MCP4921	Stm32f407
CS	PA4(CS)
SCLK	PA5(SCLK)
SDI	PA7(MOSI)

MCP4921	BUZZER
GND	GND
VOUT	IN

پس از تنظیمات لازم برای SPI کد را در محیط keil مشابه کد ذیل اصلاح نمایید.

```
#include "main.h"
#include "stdio.h"

SPI_HandleTypeDef hspi1;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);

void MCP4921_SendData(uint8_t control, uint16_t data) {
    uint8_t txData[2];

    txData[0] = (control & 0x0F) << 4 | ((data >> 8) & 0x0F);
    txData[1] = data & 0xFF;

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, txData, 2, 200);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
}

int main(void)
{
    HAL_Init();

    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();

    while (1)
    {

        MCP4921_SendData(0x07, 0x0FFF);
        HAL_Delay(500);

        MCP4921_SendData(0x07, 0x0000);
        HAL_Delay(500);

    }
}
```

}